5 *Notification*—Using so-called receivers, define how the notification should be delivered, such as via email, Slack, PagerDuty, or Amazon Simple Notification Service (SNS; https://aws.amazon.com/sns/).

An example config (http://mng.bz/MBaQ) for the Alertmanager should give you some idea of how you can configure the notification pipeline, but let's have a look at a concrete end-to-end example to fully appreciate how things work.

In this example, we implement a simple Prometheus self-monitoring case. The goal is to receive a notification once a certain number of Prometheus API calls is exceeded.

As usual, we need to stand up a Prometheus and an Alertmanager instance using the Docker Compose file in listing 7.1, which you launch (from `ch07/prometheus/`) with the following command:

```
docker-compose -f docker-compose.yaml up
```

In the Docker Compose file, we pass on the environment variables and the configuration files.

Listing 7.1 The Docker Compose configuration for the Prometheus alerting setup

```
version: "3.5"
services:

  prometheus:                          # Configures Prometheus with alerts and Alertmanager
    image: prom/prometheus:latest
    ports:
    - "9090:9090"
    volumes:
    - ./prom-config.yaml:/etc/prometheus/prometheus.yml
    - ./alerting.yaml:/etc/prometheus/alert.rules

  alertmanager:                        # Configures Alertmanager with grouping and notification
    image: prom/alertmanager:latest
    volumes:
    - ./am-config.yaml:/etc/alertmanager/alertmanager.yml
    ports:
    - "9093:9093"

  webhook:                             # Sets up a generic webhook the Alertmanager can call
    image: kennethreitz/httpbin
    ports:
    - "9999:80"
```

The next listing shows the complete Prometheus configuration in prometheus.yml, defining where to find alerts and connecting to the Alertmanager.

Listing 7.2 The Prometheus config

```
global:
  scrape_interval: 5s
```